

About One-Sided One-Symbol Insertion-Deletion P Systems

Sergiu Ivanov¹ and Sergey Verlan^{1,2}

¹ Laboratoire d'Algorithmique, Complexité et Logique,
Université Paris Est – Créteil Val de Marne,
61, av. gén. de Gaulle, 94010 Créteil, France
email: {sergiu.ivanov,verlan}@u-pec.fr

² Institute of Mathematics and Computer Science,
Academy of Sciences of Moldova,
Academiei 5, Chisinau, MD-2028, Moldova

Abstract. In this article we consider insertion-deletion P systems inserting or deleting one symbol in one or two symbol(s) left context (more precisely of size $(1, 2, 0; 1, 1, 0)$ and $(1, 1, 0; 1, 2, 0)$). We show that computational completeness can be achieved by using only 3 membranes in a tree-like structure. Hence we obtain a trade-off between the sizes of contexts of insertion and deletion rules and the number of membranes sufficient for computational completeness.

1 Introduction

The operations of insertion and deletion were first considered with a linguistic motivation [19, 8, 22]. Another inspiration for these operations comes from the fact that the insertion operation and its iterated variants are generalized versions of Kleene's operations of concatenation and closure [14], while the deletion operation generalizes the quotient operation. A study of properties of the corresponding operations may be found in [10–12]. However, insertion and deletion also have interesting biological motivations, e.g., they correspond to a mismatched annealing of DNA sequences; these operations are also present in the evolution processes in the form of point mutations as well as in RNA editing, see the discussions in [3, 4, 26] and [24]. These biological motivations of insertion-deletion operations led to their study in the framework of molecular computing, see, for example, [6, 13, 24, 27].

In general, an insertion operation means adding a substring to a given string in a specified (left and right) context, while a deletion operation means removing a substring of a given string from a specified (left and right) context. A finite set of insertion-deletion rules, together with a set of axioms provide a language generating device: starting from the set of initial strings and iterating insertion-deletion operations as defined by the given rules, one gets a language.

Even in their basic variants, insertion-deletion systems are able to characterize the recursively enumerable languages. Moreover, as it was shown in [20], the context dependency may be replaced by insertion and deletion of strings of

sufficient length, in a context-free manner. If the length is not sufficient (less or equal to two) then such systems are decidable and a characterization of them was shown in [28].

Similar investigations were continued in [21, 16, 17] on insertion-deletion systems with one-sided contexts, i.e., where the context dependency is present only from the left or only from the right side of all insertion and deletion rules. The papers cited above give several computational completeness results depending on the size of parameters of insertion and deletion rules. We recall the interesting fact that some combinations are not leading to computational completeness, i.e., there are recursively enumerable languages that cannot be generated by such devices, in particular, by systems of size $(1, 1, 0; 1, 1, 0)$, where the first three numbers represent the maximal size of the inserted string and the maximal size of the left and right contexts, respectively, while the last three numbers provide the same information about deletion rules.

In order to increase the computational power of the corresponding variants they were considered in the framework of P systems [18] and it was shown that computational completeness can be achieved if 5 membranes are used. In [7] tissue P systems are considered and computational completeness is achieved with 4 membranes. In [2] computational completeness is achieved by simpler insertion-deletion rules, but instead using priorities. A summary of related results can be found in [1, 29].

In this article we would like to consider the trade-offs between the sizes of the contexts and the number of membranes. We consider insertion-deletion P systems of size $(1, 2, 0; 1, 1, 0)$ and $(1, 1, 0; 1, 2, 0)$, and show that computational completeness can be achieved with only 3 membranes. We remind that previously it was shown that 4 membranes are enough to achieve computational completeness with insertion and deletion rules of size $(1, 1, 0)$.

2 Preliminaries

In this paper, the empty string is denoted by λ , the family of recursively enumerable, context-sensitive, and context-free languages by RE , CS and CF , respectively. We will use the notation $|w|$ for the length of a string w , while the number of occurrences of the symbol a in the string w will be referred to by the notation $|w|_a$. We do not define the standard concepts of the theory of formal languages in this section; the reader is invited to consider [25] for further details.

A type-0 grammar $G = (N, T, S, P)$ is said to be in *Geffert normal form* [9] if the set of non-terminals N is defined as $N = \{S, A, B, C, D\}$, T is an alphabet and P only contains context-free rules of the forms $S \rightarrow uSv$ with $u \in \{A, C\}^+$ and $v \in (T \cup \{B, D\})^+$ as well as $S \rightarrow \lambda$ and two (non-context-free) erasing rules $AB \rightarrow \lambda$ and $CD \rightarrow \lambda$.

We remark that according to [9] the generation of a string using a grammar in this normal form is done in two stages. During the first stage only context-free rules $S \rightarrow uSv$ can be applied (this follows from the fact that $u \in \{A, C\}^+$ and $v \in (\{B, D\} \cup T)^+$). During the second stage only non-context-free rules can

be applied (because there is no more symbol S in the string). The transition between the stages is done by the rule $S \rightarrow \lambda$ (note that in [9] a set of rules of the form $S \rightarrow uv$ is used instead leading to an equivalent result). Note that the symbols A, B, C, D are treated like terminals during the first stage and so, each rule $S \rightarrow uSv$ is in some sense “linear”.

Throughout this paper we will use the special Geffert normal form. Let $G = (N, T, S, P)$ be a grammar with $N = N' \cup N''$, $N' \cap N'' = \emptyset$, where $N'' = \{A, B, C, D\}$ and N' is a set of non-terminals containing S , S' and some other auxiliary non-terminals (that are introduced by the translation from the Geffert normal form to the special variant). We say that G is in the *special Geffert normal form* if it only has two (non-context-free) erasing rules $AB \rightarrow \lambda$ and $CD \rightarrow \lambda$ and several context-free rules of one of the following forms:

$$\begin{aligned} X &\rightarrow bY, & \text{where } X, Y \in N', b \in N'', X \neq Y \\ X &\rightarrow Yb, & \text{where } X, Y \in N', b \in T \cup N'', X \neq Y \\ S' &\rightarrow \lambda. \end{aligned}$$

Moreover, it may be assumed without loss of generality that for any two rules $X \rightarrow w$ and $U \rightarrow w$ in P with the first symbol of w different from S, S' , we have $U = X$.

Any grammar G in the Geffert normal form can be transformed into a grammar G' in the special Geffert normal form generating the same language by replacing the “linear” rules by right- and left-linear ones. Let S' be a new non-terminal that will be used to mark the transition from the first stage to the second. The rule $S \rightarrow uSv$ of G , where $u = a_1 \dots a_n$ and $v = b_1 \dots b_m$ is replaced in G' by the following rules: $S \rightarrow a_1 X_1$, $X_1 \rightarrow a_2 X_2$, $\dots$, $X_{n-1} \rightarrow a_n X_n$, $X_n \rightarrow X_{n+1} b_m$, $\dots$, $X_{n+m} \rightarrow S b_1$, where $X_1, \dots, X_{n+m}$ are new non-terminals different from each other as well as from the corresponding non-terminals introduced by the translation of other rules. We also add rules $X_{n+m} \rightarrow S' b_m$ and $S' \rightarrow \lambda$ to G' in order to mark the transition to the second stage. Note that the rule $S \rightarrow \lambda$ is not preserved in G' .

We also note that during the first stage of the derivation of a grammar in the special Geffert normal form there is exactly one non-terminal from N' present in the string and during the second stage the string does not contain any symbol from N' .

An *insertion-deletion system* is a construct $\Gamma = (V, T, A, I, D)$, where V is an alphabet, $T \subseteq V$ is the *terminal* alphabet (the symbols from $V \setminus T$ are called *non-terminal* symbols), $A \subseteq V^*$ is the set of *axioms*, and I and D are finite sets of triples of the form (u, α, v) , where u, α , and v are strings over V , with $\alpha \neq \lambda$. The triples in I are called *insertion rules*, and those in D are called *deletion rules*.

An insertion rule $(u, \alpha, v) \in I$ indicates that the string α can be inserted between u and v , while a deletion rule $(u, \alpha, v) \in D$ indicates that α can be removed from between the contexts u and v . In other words, $(u, \alpha, v) \in I$ corresponds to the rewriting rule $uv \rightarrow u\alpha v$, while $(u, \alpha, v) \in D$ corresponds to the rewriting rule $u\alpha v \rightarrow uv$.

We denote the “derives by insertion” relation induced by insertion rules by $\Rightarrow_{ins}$. Formally, $x \Rightarrow_{ins} y$ (“ x derives y by insertion”) if and only if $x = x_1uvx_2$ and $y = x_1u\alpha vx_2$, $x_1, x_2 \in V^*$, and there exists $(u, \alpha, v) \in I$. By the notation $\Rightarrow_{del}$ we refer to the “derives by deletion” relation defined by deletion rules. Formally, $x \Rightarrow_{del} y$ (“ x derives y by deletion”) if and only if $x = x_1u\alpha vx_2$ and $y = x_1uvx_2$, $x_1, x_2 \in V^*$, and there exists $(u, \alpha, v) \in D$. By $\Rightarrow$ we refer to the union of the relations $\Rightarrow_{ins}$ and $\Rightarrow_{del}$, and by $\xRightarrow{*}$ we denote the reflexive and transitive closure of $\Rightarrow$.

Instead of relying on separate sets I and D , we will often consider their union $R = I \cup D$ and distinguish between insertion and deletion rules by the subscripts ins and del . Thus instead of $(u, \alpha, v) \in I$, we will write $(u, \alpha, v)_{ins}$, and instead of $(u, \alpha, v) \in D$, we will write $(u, \alpha, v)_{del}$.

The language generated by the insertion-deletion system $\Gamma = (V, T, A, I, D)$ is defined as follows:

$$L(\Gamma) = \{w \in T^* \mid x \xRightarrow{*} w, x \in A\}.$$

The complexity of an insertion-deletion system $\Gamma = (V, T, A, I, D)$ is described by the vector $(n, m, m'; p, q, q')$ called *size*, where

$$\begin{aligned} n &= \max\{|\alpha| \mid (u, \alpha, v) \in I\}, & p &= \max\{|\alpha| \mid (u, \alpha, v) \in D\}, \\ m &= \max\{|u| \mid (u, \alpha, v) \in I\}, & q &= \max\{|u| \mid (u, \alpha, v) \in D\}, \\ m' &= \max\{|v| \mid (u, \alpha, v) \in I\}, & q' &= \max\{|v| \mid (u, \alpha, v) \in D\}. \end{aligned}$$

The *total size* of an insertion-deletion system Γ of size $(n, m, m'; p, q, q')$ is defined as the sum of all the numbers from the vector: $\Sigma(\Gamma) = n + m + m' + p + q + q'$.

By $INS_n^{m, m'} DEL_p^{q, q'}$ we denote the families of languages generated by insertion-deletion systems of size $(n, m, m'; p, q, q')$.

If $*$ is specified instead of one of the parameters n, m, m', p, q , or q' , then there are no restrictions on the length of the corresponding component. In particular, $INS_*^{0, 0} DEL_*^{0, 0}$ denotes the family of languages generated by context-free insertion-deletion systems.

If one of the numbers from the pairs m, m' or q, q' is equal to zero, while the other one is not, we say that the family is with one-sided context.

An *insertion-deletion P system* of degree n is the following construct:

$$\Pi = (V, T, \mu, M_1, \dots, M_n, R_1, \dots, R_n)$$

where

- V is a finite alphabet,
- $T \subseteq V$ is the terminal alphabet,
- μ is the membrane (tree) structure of the system which has n membranes (nodes). This structure will be represented by a word containing correctly nested marked parentheses.
- M_i , for each $1 \leq i \leq n$ is a finite language associated with the membrane i .

- R_i , for each $1 \leq i \leq n$ is a set of insertion and deletion rules with target indicators associated with membrane i and having the following forms: $(u, x, v; tar)_{ins}$, where (u, x, v) is an insertion rule, and $(u, x, v; tar)_{del}$, where (u, x, v) is a deletion rule, and tar , called the *target indicator*, is from the set $\{here, in, out\}$.

Any n -tuple $(N_1, \dots, N_n)$ of languages over V is called a configuration of Π . For two configurations $(N_1, \dots, N_n)$ and $(N'_1, \dots, N'_n)$ of Π we write $(N_1, \dots, N_n) \Rightarrow (N'_1, \dots, N'_n)$ if one can pass from $(N_1, \dots, N_n)$ to $(N'_1, \dots, N'_n)$ by applying the insertion and deletion rules from each region of μ , in the maximally parallel way, i.e., in parallel to all possible strings from the corresponding regions, and following the target indications associated with the rules. We assume that every string represented in a membrane has arbitrary many copies. Hence, by applying a rule to a string we get both arbitrary many copies of resulting string as well as old copies of the same string.

More specifically, if $w \in N_i$ and $r = (u, x, v; tar)_{ins} \in R_i$, respectively $r = (u, x, v; tar)_{del} \in R_i$, such that $w \xRightarrow{r}_{ins} w'$, respectively $w \xRightarrow{r}_{del} w'$, then w' will go to the region indicated by tar . If $tar = here$, then the string remains in N_i , if $tar = out$, then the string is moved to the region immediately outside the membrane i (maybe, in this way the string leaves the system), if $tar = in$, then the string is moved to one of the regions immediately below region j .

A sequence of transitions between configurations of a given insertion-deletion P system Π , starting from the initial configuration $(M_1, \dots, M_n)$, is called a computation with respect to Π . The result of a computation consists of all strings over T which are sent out of the system at any time during the computation. We denote by $L(\Pi)$ the language of all strings of this type. We say that $L(\Pi)$ is generated by Π .

As in [23] we denote by $ELSP_k(ins_n^{m,m'}, del_p^{q,q'})$ the family of languages generated by insertion-deletion P systems of degree at most $k \geq 1$ having the size $(n, m, m'; p, q, q')$.

3 Computational power of one-sided insertion-deletion systems of small size

In this section we consider insertion-deletion P systems of size $(1, 2, 0; 1, 1, 0)$ and $(1, 1, 0; 1, 2, 0)$. While the computational power of normal insertion-deletion systems with these parameters is not yet known, based on observations from [15] we conjecture that the corresponding models are not computationally complete. We also recall that most combinations of parameters involving left and right contexts as well as the insertion or deletion of more than one symbol are known to produce computationally complete insertion-deletion systems, see [29] for a complete list.

Theorem 1 $ELSP_3(ins_1^{2,0}, del_1^{1,0}) = RE$.

Proof. Consider a type-0 grammar $G = (N, T, P, S)$ in the special Geffert normal form and let $N'' = \{A, B, C, D\} \subseteq N$. We construct an insertion-deletion P system

$$\Pi = (V, T, [1[2[3]3]2]_1, \{\{\mathcal{X}S\}\}, \emptyset, \emptyset, R_1 \cup R'_1, R_2, R_3)$$

that simulates G as follows. The rules from P are supposed to be labeled in a one-to-one manner with labels from the set $[1..|P|]$. The alphabet of Π is $V = N \cup T \cup \{M_i \mid i : X \rightarrow \alpha \in P\} \cup \{K, K', \mathcal{X}\}$. The sets of rules R_1, R_2, R_3 of Π are defined as follows.

For any rule $i : X \rightarrow bY \in P$ we consider following sets of rules:

$$\begin{aligned} R_1^i &= \{i.1 : (X, M_i, \lambda; in)_{ins}\}, \\ R_2^i &= \{i.2 : (XM_i, Y, \lambda; in)_{ins}\} \cup \{i.3 : (\mathbf{a}, M_i, \lambda; out)_{del} \mid \mathbf{a} \in N''\}, \\ R_3^i &= \{i.4 : (\mathbf{a}, X, \lambda; here)_{del} \mid \mathbf{a} \in N''\} \cup \{i.5 : (\mathbf{a}M_i, b, \lambda; out)_{ins} \mid \mathbf{a} \in N''\}. \end{aligned}$$

For any rule $i : X \rightarrow Yb$ we consider following sets of rules:

$$\begin{aligned} R_1^i &= \{i.1 : (X, M_i, \lambda; in)_{ins}\}, \\ R_2^i &= \{i.2 : (XM_i, b, \lambda; in)_{ins}\} \cup \{i.3 : (\mathbf{a}, M_i, \lambda; out)_{del} \mid \mathbf{a} \in N''\}, \\ R_3^i &= \{i.4 : (\mathbf{a}, X, \lambda; here)_{del} \mid \mathbf{a} \in N''\} \cup \{i.5 : (\mathbf{a}M_i, Y, \lambda; out)_{ins} \mid \mathbf{a} \in N''\}. \end{aligned}$$

For the rules $i_1 : AB \rightarrow \lambda$ and $i_2 : CD \rightarrow \lambda$ and $i_3 : S' \rightarrow \lambda$ we consider following sets of rules:

$$\begin{aligned} R_1^{i_1} &= \{i_1.1 : (\lambda, K, \lambda; in)_{ins}\}, \\ R_2^{i_1} &= \{i_1.2 : (K, A, \lambda; in)_{del}\} \cup \{i_1.3 : (\lambda, K, \lambda; out)_{del}\}, \\ R_3^{i_1} &= \{i_1.4 : (K, B, \lambda; out)_{del}\}, \\ R_1^{i_2} &= \{i_2.1 : (\lambda, K', \lambda; in)_{ins}\}, \\ R_2^{i_2} &= \{i_2.2 : (K', C, \lambda; in)_{del}\} \cup \{i_2.3 : (\lambda, K, \lambda; out)_{del}\}, \\ R_3^{i_2} &= \{i_2.4 : (K', D, \lambda; out)_{del}\}, \\ R_1^{i_3} &= \{i_3.1 : (\lambda, S', \lambda; here)_{del}\}. \end{aligned}$$

Now for $j = 1, 2, 3$ we define $R_j = \cup_{1 \leq i \leq |P|} R_j^i$ and we define $R'_1 = \{\mathcal{X} : (\lambda, \mathcal{X}, \lambda; out)_{del}\}$.

We state that $L(\Pi) = L(G)$. For this we show how each rule of G can be simulated in Π . Consider a string wXw' in membrane 1 and suppose that there is a rule $i : X \rightarrow bY$ in P . Then the following unique evolution can happen:

$$\begin{aligned} (wXw', 1) &\Longrightarrow_{i.1} (wXM_iw', 2) \Longrightarrow_{i.2} (wXM_iYw', 3) \Longrightarrow_{i.4} \\ &\Longrightarrow_{i.4} (wM_iYw', 3) \Longrightarrow_{i.5} (wM_ibYw', 2) \Longrightarrow_{i.3} (wbYw', 1). \end{aligned}$$

In the second step it was possible to apply the rule $i.3$, yielding string wXw' in membrane 1, but this just returns to the previous configuration.

The rule $X \rightarrow Yb$ is simulated in a similar manner:

$$\begin{aligned} (wXw', 1) &\Rightarrow_{i.1} (wXM_iw', 2) \Rightarrow_{i.2} (wXM_ibw', 3) \Rightarrow_{i.4} \\ &\Rightarrow_{i.4} (wM_ibw', 3) \Rightarrow_{i.5} (wM_iYbw', 2) \Rightarrow_{i.3} (wYbw', 1). \end{aligned}$$

The rule $i_1 : AB \rightarrow \lambda$ is simulated as follows (the case of rule $i_2 : CD \rightarrow \lambda$ is treated in an analogous way). First a symbol K is inserted in a context-free manner into the string ww' by using the rule $i_1.1$, yielding wKw' . If the symbol to the right of K is not an A , then the only possibility is to apply rule $i_1.3$ which deletes K and returns the string ww' to membrane 1. If K is inserted in front of a symbol A ($w' = Aw''$) then rule $i_1.2$ can be applied and string wKw'' goes to membrane 3. Now if w'' does not start with B , then the computation of this word is stopped and it does not yield a result. Otherwise ($w'' = Bw'''$), rule $i_1.4$ is applied yielding wKw''' in membrane 2. Now the computation may be continued in the same manner and K either eliminates another couple of symbols AB if this is possible, or the string appears in the skin membrane without K and then is ready for new evolutions.

When the system Π reaches the configuration $\mathcal{X}w$ with $w \in T^*$, rule $\mathcal{X}$ from R'_1 can be applied yielding a terminal string w in the environment as a result of the computation.

Now in order to complete the proof, we observe that the only sequences of rules leading to a terminal derivation in Π correspond to the groups of rules as defined above. Hence, a derivation in G can be reconstructed from a derivation in Π . $\square$

Theorem 2 $ELSP_3(ins_1^{1,0}del_1^{2,0}) = RE$.

Proof. Consider the type-0 grammar $G = (N, T, S, P)$ in the special Geffert normal form and denote $N'' = \{A, B, C, D\} \subseteq N$. Consider as well that the rules from P are bijectively labelled with the numbers from the set $[1..|P|]$. We will now construct the following insertion-deletion P system Π which simulates G :

$$\Pi = (V, T, [_1[_2[_3]_3]_2]_1, \{\{\mathcal{X}S\}\}, \emptyset, \emptyset, R_1 \cup R'_1, R_2 \cup R'_2, R_3 \cup R'_3).$$

The set of objects of Π will contain new special symbols per each rule of G and is constructed in the following way:

$$\begin{aligned} V = & \{M_i, \bar{Y}_i, M'_i \mid i : X \rightarrow bY \in P\} \\ & \cup \{M_i, N_i, \bar{Y}_i, M'_i \mid i : X \rightarrow Yb \in P\} \\ & \cup \{K, K', \mathcal{X}\} \cup N \cup T. \end{aligned}$$

For each $i : X \rightarrow bY \in R$, we construct the following three sets of rules:

$$\begin{aligned} R_1^i &= \{i.1 : (\lambda, M_i, \lambda; in)_{ins}\}, \\ R_2^i &= \{i.2 : (M_i, \bar{Y}_i, \lambda; here)_{ins}\} \cup \{i.3 : (M_i, b, \lambda; in)_{ins}\} \\ &\quad \cup \{i.4 : (b\bar{Y}_i, X, \lambda; out)_{del}\}, \\ R_3^i &= \{i.5 : (\lambda, M_i, \lambda; out)_{del}\}. \end{aligned}$$

For each $i : X \rightarrow Yb \in R$, we construct the following three sets of rules:

$$\begin{aligned} R_1^i &= \{i.1 : (\lambda, M_i, \lambda; in)_{ins}\}, \\ R_2^i &= \{i.2 : (M_i, N_i, \lambda; here)_{ins}\} \cup \{i.3 : (N_i, b, \lambda; in)_{ins}\} \\ &\quad \cup \{i.4 : (\bar{Y}_i b, X, \lambda; out)_{del}\}, \\ R_3^i &= \{i.5 : (M_i, \bar{Y}_i, \lambda; here)_{ins}\} \cup \{i.6 : (\lambda, M_i, \lambda; here)_{del}\} \\ &\quad \cup \{i.7 : (\mathbf{a}\bar{Y}_i, N_i, \lambda; out)_{del} \mid \mathbf{a} \in N''\}. \end{aligned}$$

Moreover, we also build the following three sets:

$$\begin{aligned} R_1' &= \{i_1' : (\lambda, M_i', \lambda; in)_{ins} \mid i : X \rightarrow bY \in P \text{ or } i : X \rightarrow Yb \in P\}, \\ R_2^i &= \{i_2' : (M_i', Y, \lambda; in)_{ins} \mid i : X \rightarrow bY \in P \text{ or } i : X \rightarrow Yb \in P\} \\ &\quad \cup \{i_3' : (\lambda, M_i', \lambda; in)_{del} \mid i : X \rightarrow bY \in P \text{ or } i : X \rightarrow Yb \in P\}, \\ R_3^i &= \{i_4' : (M_i'Y, \bar{Y}_i, \lambda; out)_{del} \mid i : X \rightarrow bY \in P \text{ or } i : X \rightarrow Yb \in P\}. \end{aligned}$$

Finally, for the rules $i_1 : AB \rightarrow \lambda$, $i_2 : CD \rightarrow \lambda$, and $i_3 : S' \rightarrow \lambda$ we consider the following sets of rules:

$$\begin{aligned} R_1^{i_1} &= \{i_1.1 : (\lambda, K, \lambda; in)_{ins}\}, \\ R_2^{i_1} &= \{i_1.2 : (K, A, \lambda; in)_{del}\} \cup \{i_1.3 : (\lambda, K, \lambda; out)_{del}\}, \\ R_3^{i_1} &= \{i_1.4 : (K, B, \lambda; out)_{del}\}, \\ R_1^{i_2} &= \{i_2.1 : (\lambda, K', \lambda; in)_{ins}\}, \\ R_2^{i_2} &= \{i_2.2 : (K', C, \lambda; in)_{del}\} \cup \{i_2.3 : (\lambda, K, \lambda; out)_{del}\}, \\ R_3^{i_2} &= \{i_2.4 : (K', D, \lambda; out)_{del}\}, \\ R_1^{i_3} &= \{i_3.1 : (\lambda, S', \lambda; here)_{del}\}. \end{aligned}$$

Now for $j = 1, 2, 3$ we define the sets $R_j = \cup_{1 \leq i \leq |P|} R_j^i$ and also $R_1' = \{\mathcal{X} : (\lambda, \mathcal{X}, \lambda; out)_{del}\}$.

We state that $L(\Pi) = L(G)$. For this we show how each rule of G can be simulated in Π . Consider a string wXw' in membrane 1 and suppose that there is a rule $i : X \rightarrow bY$ in P . The simulation of this rule occurs in two phases: in the first phase we rewrite X to $b\bar{Y}_i$, while in the second one we substitute $\bar{Y}_i$ with Y . The following is the valid first-phase simulation sequence in Π :

$$\begin{aligned} (wXw', 1) &\Rightarrow_{i.1} (wM_iXw', 2) \Rightarrow_{i.2} (wM_i\bar{Y}_iXw', 2) \Rightarrow_{i.3} (wM_i b\bar{Y}_iXw', 3) \\ &\Rightarrow_{i.5} (wb\bar{Y}_iXw, 2) \Rightarrow_{i.4} (wb\bar{Y}_i w', 1). \end{aligned}$$

The second phase happens due to the rules in the sets R_i' , $i = 1, 2, 3$, and consists of the following steps:

$$\begin{aligned} (w\bar{Y}_i bw', 1) &\Rightarrow_{i'.1} (wM_i' \bar{Y}_i bw', 2) \Rightarrow_{i'.2} (wM_i' Y \bar{Y}_i bw', 3) \\ &\Rightarrow_{i'.4} (wM_i' Y bw', 2) \Rightarrow_{i'.3} (wY bw', 1). \end{aligned}$$

We claim the both the first phase and the second phase simulation sequences are the only ones which can happen in valid derivations of Π . Indeed, consider

the wXw' into which $i.1$ has inserted an instance of M_i . By inspecting the symbol requirements of the rules associated with membrane 2, we conclude that only the rules $i.2$ and $i.3$ may become applicable. Suppose that rule $i.3$ is applied directly. If, for example, M_i has been inserted to the right of X , this will produce the string $\gamma M_i b \gamma'' X w'$, which will be moved into membrane 3. The case when $i.1$ inserts M_i to the left of X is treated in a similar way. Now, the only way to further move the computation out of membrane 3 is by applying the rule $i.5$ which will remove the instance of M_i and move the string into the second membrane. However, no more rules will be applicable from now on, because the string contains no service symbols at all, but is in the second membrane.

Suppose now that, after the application of $i.1$, the rule $i.2$ is applied $k > 1$ times. The subsequent application of the rule $i.3$ will insert an instance of b after M_i , thus yielding the substring $M_i b (\bar{Y}_i)^k$. Again, the only way to move the string out of membrane 3 is to erase the symbol M_i which produces a string with a substring of k instances of $\bar{Y}_i$. It is clear that, if X is situated to the left of this $(\bar{Y}_i)^k$, the string cannot contain $\bar{Y}_i X$, which is required by $i.3$. On the other hand, if X is to the right of $(\bar{Y}_i)^k$, it will not be possible to apply $i.3$ again, because the string does not contain the substring $\bar{Y}_i X$ preceded by a symbol from N'' .

Finally, it is rather clear that, if $i.1$ does not insert the M_i just to the left of X , Π will not be able to move the string containing a $\bar{Y}_i$ and an X out of membrane 2, thus blocking without producing any meaningful result.

We will focus on the second-phase simulation sequence now. The application of the rule $i'.1$ inserts an instance of M'_i somewhere and moves the string into membrane 2. There are only two rules that may become applicable: $i'.2$ and $i'.3$. Suppose that $i'.3$ is applied directly after $i'.1$. In this case the system will come back into the configuration it has been in before the application of $i'.1$ without doing any changes to the string whatsoever. Therefore, to actually modify the string, the rule $i'.2$ must be applied.

An application of the rule $i'.2$ inserts exactly one instance of Y after M'_i and puts the string into the innermost membrane 3. Now, the only way to exit this membrane is by applying the rule $i'.4$, which means that, if the application of the rule $i'.1$ has not inserted M'_i to the left of $\bar{Y}_i$, the system Π will unproductively block in the third membrane. Consequently, after the application of $i'.4$, the string in the second membrane must be of the form $wM'_i Y b w'$. At this point, two rules are still applicable, $i'.2$ and $i'.3$. Suppose indeed that the rule $i'.2$ is applied a second time and inserts another instance of Y after M'_i , thus yielding the string $wM'_i Y Y b w'$ and moving it into membrane 3. Now, however, the rule $i'.4$ is not applicable because the string lacks $\bar{Y}_i$ and Π will thus block. Therefore, the only productive way to move the string $wM'_i Y b w'$ out of the second membrane is to apply $i'.3$.

Now consider a rule $i : X \rightarrow Yb$. Again, the simulation of i happens in two phases: in the first phase we rewrite X to $\bar{Y}_i b$, while in the second phase we substitute $\bar{Y}_i$ with Y . Since the second phase of the simulation happens in exactly the same way as in the case of the rule $X \rightarrow bY$, we will only focus on

the first-stage simulation sequence:

$$\begin{aligned} (wXw', 1) &\Rightarrow_{i.1} (wM_iXw', 2) \Rightarrow_{i.2} (wM_iN_iXw', 2) \Rightarrow_{i.3} (wM_iN_ibXw', 3) \\ &\Rightarrow_{i.5} (wM_i\bar{Y}_iN_ibXw', 3) \Rightarrow_{i.6} (w\bar{Y}_iN_ibXw', 3) \\ &\Rightarrow_{i.7} (w\bar{Y}_ibXw', 2) \Rightarrow_{i.4} (w\bar{Y}_ibw', 1). \end{aligned}$$

We claim that the first-phase simulation sequence we have just shown is the only possible valid derivation of Π . We will now consider the variations that can interfere with this subderivation and show that none of them can influence the result of a computation of Π .

Consider the application of $i.1$ which inserts M_i into the original string wXw' and moves the new string, say $\gamma M_i \gamma' Xw'$, into membrane 2. The case when M_i is inserted to the right of X is treated in a similar way. In the current situation, the only applicable rule is $i.3$, which may insert k instances of N_i , thus yielding the string $\gamma M_i (N_i)^k \gamma' Xw'$. If one discards the possibility to produce yet more instances of N_i , the only other way to evolve is the application of the rule $i.3$ to insert a b after one of the N_i 's and thereby to move the string into membrane 3.

In the new configuration, membrane 3 will contain $\gamma M_i (N_i)^{k_1} b (N_i)^{k_2} \gamma' Xw'$, where $k_1 \geq 1$ and $k_1 + k_2 = k$. We immediately remark that the only way for Π to move out of this membrane is to apply the rule $i.7$. This rule requires that there is a substring of $\bar{Y}_i N_i$ preceded by a symbol from N'' . The string $\gamma M_i (N_i)^{k_1} b (N_i)^{k_2} \gamma' Xw'$, with which the system Π has just arrived in membrane 3, does not contain any instances of $\bar{Y}_i$, but the rule $i.5$ can introduce them. Suppose this latter rule is applied t times, $t \geq 0$, thus yielding the following result:

$$\gamma M_i (\bar{Y}_i)^t (N_i)^{k_1} b (N_i)^{k_2} \gamma' Xw'.$$

Clearly, the rule $i.7$ is not yet applicable, because there are no instances of $\bar{Y}_i$ preceded by symbols from N'' . The only way to reach this situation is to apply the rule $i.6$ to obtain the string

$$\gamma (\bar{Y}_i)^t (N_i)^{k_1} b (N_i)^{k_2} \gamma' Xw'.$$

The rule $i.7$ imposes an even stronger requirement: the instance of $\bar{Y}_i$ which is preceded by a symbol from N'' must be immediately followed by N_i . Since instances of $\bar{Y}_i$ can only be inserted to the right of M_i , and since the process of inserting N_i 's has already been completed in membrane 2, applying $i.7$ actually requires that exactly one instance of $\bar{Y}_i$ has been inserted by $i.5$ (i.e., it requires that $t = 1$), giving

$$\gamma \bar{Y}_i (N_i)^{k_1} b (N_i)^{k_2} \gamma' Xw'.$$

An application of the rule $i.7$ will erase the leftmost instance of N_i and will put the following string into membrane 2:

$$\gamma \bar{Y}_i (N_i)^{k_1-1} b (N_i)^{k_2} \gamma' Xw'.$$

The rule $i.3$ will still be applicable at this moment. Remark, however, that the string which will be moved into membrane 3 by this application will contain no

instances of M_i , so the rule which may be applicable is $i.7$, which will remove yet another instance of N_i following $\bar{Y}_i$. Applications of the rules $i.3$ and $i.7$ in a loop will only be possible as long as there are instances of N_i just to the right of $\bar{Y}_i$ and then Π will either block in membrane 3 or move the string into membrane 1 with an application of $i.4$.

Based on the observations we have made in the previous paragraph, we can assert that the general form of the strings which may appear in membrane 2 after at least one traversal of membrane 3 is $\gamma\bar{Y}_i(N_i^*(N_ib)^*)^*\gamma'Xw'$. If we discard the possibility of yet again re-tracing the loop formed by the rules $i.3$ and $i.7$, the only other way for Π to proceed is to apply $i.4$ and move the string into membrane 1. However, the rule $i.4$ imposes a strong condition on the form of the string it can be applied to: there has to exist a substring $\bar{Y}_ibX$. Clearly, the only way to have exactly one b between $\bar{Y}_i$ and X is, firstly, to have $i.1$ insert M_i exactly to the left of X (that is, γ' should be zero) and, secondly, to only apply $i.3$ once during the whole simulation process, thus obtaining the string $\gamma\bar{Y}_ibXw'$ in membrane 2. The application of $i.4$ will thus erase the X and successfully finish the rewriting of X into $\bar{Y}_ib$.

We conclude the proof by stating the simulation of the rules $AB \rightarrow \lambda$ and $CD \rightarrow \lambda$ is done in exactly the same way as in the case of the systems from the class $ELSP_3(ins_1^{2,0}del_1^{1,0})$. $\square$

4 Conclusion

In this article we considered insertion-deletion P systems of size $(1, 2, 0; 1, 1, 0)$ and $(1, 1, 0; 1, 2, 0)$ and showed that computational completeness can be achieved with 3 membranes. Compared to [7] this result shows an interesting trade-off between the size of contexts in insertion-deletion rules and the number of membranes: with 4 membranes, computational completeness is obtained already with insertion and deletion rules of size $(1, 1, 0)$. Now it remains an open question if the number of membranes can be further decreased for the investigated systems or for systems having bigger contexts for the insertion or deletion rules.

References

1. A. Alhazov, A. Krassovitskiy, Y. Rogozhin, and S. Verlan. Small size insertion and deletion systems. In C. Martin-Vide, ed., *Scientific Applications of Language Methods*, volume 2 of *Mathematics, Computing, Language, and Life: Frontiers in Mathematical Linguistics and Language Theory*, chap. 9, 459–524. World Sci., 2010.
2. A. Alhazov, A. Krassovitskiy, Y. Rogozhin, S. Verlan, P Systems with Minimal Insertion and Deletion, *Theoretical Computer Science*, **412**(1-2), 136–144, (2011).
3. R. Benne. *RNA Editing: The Alteration of Protein Coding Sequences of RNA*. Ellis Horwood, Chichester, West Sussex, 1993.
4. F. Biegler, M. J. Burrell, and M. Daley. Regulated RNA rewriting: Modelling RNA editing with guided insertion. *Theor. Comput. Sci.*, 387(2):103 – 112, 2007.
5. E. Csuhaj-Varjú, A. Salomaa, Networks of Parallel Language Processors, *Lecture Notes in Computer Science*, 1218, 299–318, 1997.

6. M. Daley, L. Kari, G. Gloor, and R. Siromoney. Circular contextual insertions/deletions with applications to biomolecular computation. In *SPIRE/CRIWG*, 47–54, 1999.
7. R. Freund, M. Kogler, Y. Rogozhin, and S. Verlan. Graph-controlled insertion-deletion systems. In I. McQuillan and G. Pighizzini, eds, *Proc. of 12th Workshop on Descriptive Complexity of Formal Systems*, vol. 31 of *EPTCS*, 88–98, 2010.
8. B. Galiukschov. Semicontextual grammars. *Matem. Logika i Matem. Lingvistika*, 38–50, 1981. Tallin University (in Russian).
9. V. Geffert. Normal forms for phrase-structure grammars. *ITA*, 25:473–498, 1991.
10. D. Haussler. *Insertion and Iterated Insertion as Operations on Formal Languages*. PhD thesis, Univ. of Colorado at Boulder, 1982.
11. D. Haussler. Insertion languages. *Information Sciences*, 31(1):77–89, 1983.
12. L. Kari. *On Insertion and Deletion in Formal Languages*. PhD thesis, University of Turku, 1991.
13. L. Kari, G. Păun, G. Thierrin, and S. Yu. At the crossroads of DNA computing and formal languages: Characterizing RE using insertion-deletion systems. In *Proc. of 3rd DIMACS Workshop on DNA Based Computing*, 318–333. Philadelphia, 1997.
14. S. C. Kleene. Representation of events in nerve nets and finite automata. In C. Shannon and J. McCarthy, eds, *Automata Studies*, 3–41. Princeton University Press, Princeton, NJ, 1956.
15. A. Krassovitskiy. Complexity and Modeling Power of Insertion-Deletion Systems. PhD thesis, Universitat Rovira i Virgili, Tarragona, Spain, 2011.
16. A. Krassovitskiy, Y. Rogozhin, and S. Verlan. Further results on insertion-deletion systems with one-sided contexts. In C. Martín-Vide et al., eds, *Language and Automata Theory and Applications, Second International Conference, LATA 2008. Revised Papers*, vol. 5196 of *LNCS*, 333–344. Springer, 2008.
17. A. Krassovitskiy, Y. Rogozhin, and S. Verlan. Computational power of P systems with small size insertion and deletion rules. In T. Neary, D. Woods, A. K. Seda, and N. Murphy, eds, *Proc. International Workshop on The Complexity of Simple Programs, Cork, Ireland, 6-7th December 2008*, vol. 1 of *EPTCS*, 108–117, 2009.
18. A. Krassovitskiy, Y. Rogozhin, and S. Verlan. Computational power of insertion-deletion (P) systems with rules of size two. *Natural Computing*, 10(2), 835–852, 2011.
19. S. Marcus. Contextual grammars. *Rev. Roum. Math. Pures Appl.*, 14:1525–1534, 1969.
20. M. Margenstern, G. Păun, Y. Rogozhin, and S. Verlan. Context-free insertion-deletion systems. *Theor. Comput. Sci.*, 330(2):339–348, 2005.
21. A. Matveevici, Y. Rogozhin, and S. Verlan. Insertion-deletion systems with one-sided contexts. In J. O. Durand-Lose and M. Margenstern, eds, *Machines, Computations, and Universality, 5th International Conference, MCU 2007, Orléans, France, Proceedings*, vol. 4664 of *LNCS*, 205–217. 2007.
22. G. Păun. *Marcus Contextual Grammars*. Kluwer Academic Publishers, Norwell, MA, USA, 1997.
23. G. Păun. *Membrane Computing. An Introduction*. Springer-Verlag, 2002.
24. G. Păun, G. Rozenberg, and A. Salomaa. *DNA Computing: New Computing Paradigms*. Springer, 1998.
25. G. Rozenberg and A. Salomaa, editors. *Handbook of Formal Languages*. Springer-Verlag, Berlin, 1997.
26. W. D. Smith. DNA computers in vitro and in vivo. In R. Lipton and E. Baum, editors, *Proceedings of DIMACS Workshop on DNA Based Computers*, DIMACS

- Series in Discrete Math. and Theoretical Computer Science, 121–185. Amer. Math. Society, 1996.
27. A. Takahara and T. Yokomori. On the computational power of insertion-deletion systems. In M. Hagiya and A. Ohuchi, eds, *Proc. of 8th International Workshop on DNA Based Computers, Revised Papers*, vol. 2568 of *LNCS*, 269–280. 2002.
 28. S. Verlan. On minimal context-free insertion-deletion systems. *Journal of Automata, Languages and Combinatorics*, 12(1-2):317–328, 2007.
 29. S. Verlan. Study of language-theoretic computational paradigms inspired by biology. Habilitation thesis, University of Paris Est, 2010.

